(continued)

The final outcome is something like this, where matches highlighted with an HTML markup tag (`em`) indicate words that are emphasized:

```
"hits" : [
  "_source" : {
    ...
    "title" : "Effective Java",
    "synopsis" : "A must-have book for every Java programmer
  },
  "highlight" : {
    "synopsis" : [
    "A <em>must</em>-<em>have</em> <em>book</em> <em>for</em>
<em>every</em> <em>Java</em> <em>programmer</em> and Java aspirant.."]}}
]
```

We can rely on `match_phrase` for exact phrase searches. But what happens if we leave out one or two words of the phrase? For example, does the query work if we ask it to search for "must-have book Java programmer" (dropping the words for and every)? The query won't yield results if you re-run it after removing those two words. Fortunately, we can ask Elasticsearch politely by setting a `slop` parameter on the `match_phrase` query. This is discussed in the next section.

2.3.7 Phrases with missing words

The `match_phrase` query expects a full phrase: a phrase that isn't missing any words. However, users may not always input exact phrases. To handle this, Elasticsearch's solution is to set a `slop` parameter on the `match_phrase` query: a positive integer indicating how many words the phrase is missing when searching. The following query has a `slop` of 2, indicating that the search query can expect up to two words to be either missing or not in order when the query is executed.

Listing 2.13 Matching a phrase with missing words (using `slop`)

```
GET bsExecuting this query will return our book, as expected. You can
     experiment with other values of slop with the books dataset in the
     repository.
```

2.3.8 Handling spelling mistakes

Sometimes users enter incorrect spellings in their search criteria. Search engines are forgiving; they return results in spite of spelling errors. Modern search engines embrace spelling issues and provide the functionality to support them graciously. Elasticsearch works pretty hard to find similarity between the words using the *Levenshtein edit distance algorithm.*

Elasticsearch handles spelling mistakes by using a `match` query with a `fuzziness` setting. If `fuzziness` is set to 1, one spelling mistake (one letter misplaced, omitted, or added) can be forgiven. For example, if a user searches for "Komputer", by default